

Multiplicative Drift Analysis

Benjamin Doerr · Daniel Johannsen ·
Carola Winzen

Received: 31 December 2010 / Accepted: 13 February 2012 / Published online: 29 February 2012
© Springer Science+Business Media, LLC 2012

Abstract We introduce multiplicative drift analysis as a suitable way to analyze the runtime of randomized search heuristics such as evolutionary algorithms. Our multiplicative version of the classical drift theorem allows easier analyses in the often encountered situation that the optimization progress is roughly proportional to the current distance to the optimum.

To display the strength of this tool, we regard the classical problem of how the $(1 + 1)$ Evolutionary Algorithm optimizes an arbitrary linear pseudo-Boolean function. Here, we first give a relatively simple proof for the fact that any linear function is optimized in expected time $O(n \log n)$, where n is the length of the bit string. Afterwards, we show that in fact any such function is optimized in expected time at most $(1 + o(1))1.39en \ln n$, again using multiplicative drift analysis. We also prove a corresponding lower bound of $(1 - o(1))en \ln n$ which actually holds for all functions with a unique global optimum.

We further demonstrate how our drift theorem immediately gives natural proofs (with better constants) for the best known runtime bounds for the $(1 + 1)$ Evolutionary Algorithm on combinatorial problems like finding minimum spanning trees, shortest paths, or Euler tours in graphs.

Keywords Evolutionary algorithms · Randomized search heuristics · Runtime analysis · Drift analysis

Daniel Johannsen is supported by a fellowship within the Postdoc-Programme of the German Academic Exchange Service (DAAD).

Carola Winzen is a recipient of the Google Europe Fellowship in Randomized Algorithms, and this work is supported in part by this Google Fellowship.

B. Doerr · C. Winzen

Max-Planck-Institut für Informatik, Campus E1 4, 66123 Saarbrücken, Germany

D. Johannsen (✉)

School of Mathematical Sciences, Tel Aviv University, Ramat Aviv, Tel Aviv 69978, Israel

e-mail: johannse@mpi-inf.mpg.de

1 Introduction

An innocent looking problem is the question of how long the $(1 + 1)$ Evolutionary Algorithm $((1 + 1) \text{ EA})$ needs to find the optimum of a given linear function. However, this is in fact one of the problems that was most influential for the time-complexity analysis of evolutionary algorithms.

While particular linear functions like the functions ONEMAX and BINVAL were easily analyzed, it took a major effort by Droste, Jansen, and Wegener [11] to solve the problem in full generality. Their proof, however, is highly technical.

A major breakthrough spurred by this problem is the work by He and Yao [16, 18], who introduced *drift analysis* to the field of evolutionary computation. This allowed a significantly simpler proof for the linear functions problem. Even more important, it quickly became one of the most powerful tools for both proving upper and lower bounds on the expected optimization times of evolutionary algorithms [12, 13, 15, 18, 24, 25]. For a survey introducing these techniques, see Chaps. 2 and 3 in [1].

Another great progress was made by Jägersküpper [19], who combined drift analysis with a clever averaging argument to determine reasonable values for the usually not explicitly given constants. More precisely, Jägersküpper showed that the expected optimization time of the $(1 + 1)$ EA for any linear function defined on bit strings of length n is bounded from above by $(1 + o(1))2.02en \ln n$.

1.1 Classical Drift Analysis

The following method was introduced to the analysis of randomized search heuristics by He and Yao [18] and builds on a result of Hajek [14]. When analyzing the optimization behavior of a randomized search heuristic over a search space, instead of tracking how the objective function improves, one uses an auxiliary potential function and tracks its behavior.

For example, consider the search space $\{0, 1\}^n$ of bitstrings of length $n \in \mathbb{N}$.¹ Suppose we want to analyze the $(1 + 1)$ EA (which is introduced as Algorithm 1 in Sect. 3) minimizing a linear function $f: \{0, 1\}^n \rightarrow \mathbb{R}$ with

$$f(x) = \sum_{i=1}^n w_i x_i$$

and arbitrary positive weights $0 < w_1 \leq \dots \leq w_n$. (Note that we differ from previous works by always considering minimization problems. See Sect. 3 for a discussion why this does not influence the runtime analysis.) Then the potential function $h: \{0, 1\}^n \rightarrow \mathbb{R}$ can be chosen as

$$h(x) = \ln \left(1 + \sum_{j=1}^{\lfloor \frac{n}{2} \rfloor} x_j + \sum_{j=\lfloor \frac{n}{2} \rfloor + 1}^n 2x_j \right). \quad (1)$$

¹By $\mathbb{N} := \{0, 1, 2, \dots\}$ we denote the set of non-negative integers and by $\mathbb{R}$ we denote the set of real numbers.

Though still needing some calculations (see, e.g., Chap. 2 in [1]), one can show the following. Let $x \in \{0, 1\}^n$. Let $y \in \{0, 1\}^n$ be the result of one iteration (mutation and selection) of the $(1 + 1)$ EA started in x . Then there exists a $\delta > 0$ such that

$$\mathbb{E}[h(y)] \leq h(x) - \delta/n. \quad (2)$$

Now, classical drift analysis tells us that in expectation after a number of $h(x)/(\delta/n) = O(n \log n)$ iterations, the potential value is reduced to zero. But $h(x) = 0$ implies $f(x) = 0$, that is, the $(1 + 1)$ EA has found the desired optimum.

Using drift analysis to analyze a randomized search heuristic usually bears two difficulties. The first is guessing a suitable potential function h . The second, related to the first, is proving that during the search, f and h behave sufficiently similar, that is, we can prove some statement like inequality (2). Note that this inequality contains information about f as well, namely implicitly in the fact that y has an at least as good f -value as x . We see in Sect. 2.1 that although there always exists at least one potential function satisfying (2), this function may be hard to analyze.

A main difficulty in showing that h in (1) is a suitable potential function is the logarithm around the simple linear function giving weights one and two to the bits. However, since the optimization progress for linear functions is faster if we are further away from the optimum, that is, have more one-bits, this seems difficult to avoid. We present a way to ease the use of drift analysis in such settings.

1.2 Multiplicative Drift Analysis

Informally, our method applies if we have a potential function g satisfying

$$\mathbb{E}[g(y)] \leq (1 - \delta)g(x) \quad (3)$$

in the notation above. That is, we require a progress which *multiplicatively* depends on the current potential value. For this reason we call the method *multiplicative drift analysis*. We will see that for a number of problems such potential functions are a natural choice.

By formulating a drift theorem for the purpose of analyzing stochastic processes with multiplicative drift, we avoid logarithmic scaling of the potential function as in (1) and additional arguments like potential based partitions [18] or the *method of expected weight decrease* applied in [23], which seems hard to formulate in a general way. Moreover, the runtime bounds obtained by multiplicative drift analysis are often sharper than those resulting from previously used techniques.

1.3 Our Results

We apply multiplicative drift analysis, to the already mentioned problem of optimizing linear functions over $\{0, 1\}^n$. This yields a simplified proof of the $O(n \log n)$ bound on the optimization time of the $(1 + 1)$ EA. Similarly to the proof using the classical drift theorem, we make use of the simple linear function $g: \{0, 1\}^n \rightarrow \mathbb{R}$, chosen as

$$g(x) = \sum_{i=1}^n \left(1 + \frac{i}{n}\right) x_i.$$

This function g serves us as a potential function for all linear functions $f: \{0, 1\}^n \rightarrow \mathbb{R}$ with

$$f(x) = \sum_{i=1}^n w_i x_i$$

and monotone weights $0 < w_1 \leq \dots \leq w_n$.

Using parts of Jägersküpfer's analysis [19], we then improve his upper bound on the expected optimization time of the $(1 + 1)$ EA on linear functions to $(1 + o(1))1.39en \ln n$.

We also give lower bounds for this problem. We show that, in the class of all functions with a unique global optimum, the function ONEMAX (see (8)) has the smallest expected optimization time. This extends the lower bound of $(1 - o(1))en \ln n$ for the expected optimization time of the $(1 + 1)$ EA on ONEMAX [7] to all functions in that class (including all linear functions with non-zero coefficients).

Together with our upper bound, we thus obtain the remarkable result that all linear functions have roughly (within a 39% range) the same optimization time.

To further demonstrate the strength of multiplicative drift analysis, we give straightforward analyses for three prominent combinatorial problems. We consider the problems of computing minimum spanning trees (MST), single-source shortest paths (SSSP), and Euler tours. Here, we reproduce the results obtained in [23] (cf. Theorem 15), in [2] (cf. Theorem 17), and in [5] (cf. Theorem 19), respectively. In doing so, we improve the leading constants of the asymptotic bounds.

This work contains results that appeared in the preliminary version of this paper in [9] and in [10].

1.4 Subsequent Work

In the short time since its first appearance at GECCO 2010, the multiplicative drift theorem found a number of applications and extensions. Doerr and Goldberg [3] gave an alternative proof that also yields large deviation bounds. Note that, surprisingly, such bounds do not hold in the additive drift setting. In [4], the multiplicative drift theorem together with an adaptive definition of the drift function was used to prove that the $(1 + 1)$ EA with arbitrary mutation probability $p = c/n$, c a constant, finds the optimum of any linear function in time $\Theta(n \log n)$. Refining these methods, Witt [27] proved very sharp bounds for this problem, in particular, the first $(1 + o(1))en \ln(n)$ bound for the standard choice $1/n$ for the mutation probability, solving this long-standing open problem.

Going slightly beyond pseudo-Boolean linear functions, Doerr, Jansen, Sudholt, Winzen, and Zarges [8] used multiplicative drift to give a simple analysis of the $(1 + 1)$ EA optimizing arbitrary mononote functions when the mutation probability is less than $1/n$. For linear functions defined on $\{0, 1, 2\}^n$, Doerr, Johannsen and Schmidt proved a $\Theta(n \log n)$ run-time via multiplicative drift.

For combinatorial problems, multiplicative drift was successfully applied in the two following results. Doerr and Johannsen [6] gave a run-time analysis of the $(1 + 1)$ EA solving the single-source shortest path problem. Jansen, Oliveto and Zarges [20] analyzed the immune-inspired B-Cell Algorithm finding vertex covers in graphs.

2 Multiplicative Drift Analysis

Drift analysis can be used to track the optimization behavior of a randomized search heuristic over a search space by measuring the progress of the algorithm with respect to a *potential function*. Such a function maps each search point to a non-negative real number, where a potential of zero indicates that the search point is optimal.

Theorem 1 (Additive Drift [18]) *Let $S \subseteq \mathbb{R}$ be a finite set of positive numbers and let $\{X^{(t)}\}_{t \in \mathbb{N}}$ be a sequence of random variables over $S \cup \{0\}$. Let T be the random variable that denotes the first point in time $t \in \mathbb{N}$ for which $X^{(t)} = 0$.*

Suppose that there exists a real number $\delta > 0$ such that

$$\mathbb{E}[X^{(t)} - X^{(t+1)} \mid T > t] \geq \delta \quad (4)$$

holds. Then

$$\mathbb{E}[T \mid X^{(0)}] \leq \frac{X^{(0)}}{\delta}.$$

This theorem tells us how to link the expected time at which the potential reaches zero to the first time the expected value of the potential reaches zero. If in expectation the potential decreases in each step by δ then after $X^{(0)}/\delta$ steps the expected potential is zero. Thus, one might expect the expected number of steps until the (random) potential reaches zero to be $X^{(0)}/\delta$ too. This is indeed the case in the setting of the previous theorem.

In order to apply the previous theorem to the analysis of randomized search heuristics over a (finite) search space $\mathcal{S}$, we define a potential function $h: \mathcal{S} \rightarrow \mathbb{R}$ which maps all optimal search points to zero and all non-optimal search points to values strictly larger than zero. We choose the random variable $X^{(t)}$ as the potential $h(x^{(t)})$ of the search point (or population) in the t th iteration of the algorithm. Then the random variable T becomes the optimization time of the algorithm, that is, the number of iterations until the algorithm finds an optimum.

When applying Theorem 1, we call the expected difference between $h(x^{(t)})$ and $h(x^{(t+1)})$ the *drift* of the random process $\{x^{(t)}\}_{t \in \mathbb{N}}$ with respect to h . We say this drift is *additive* if condition (4) holds.

2.1 Ideal Potential Functions for Additive Drift Analysis

The application of additive drift analysis (Theorem 1) to the runtime analysis of randomized search heuristics requires a suitable potential function. The following lemma (Lemma 3 in [18]) tells us that if the random search points $x^{(0)}, x^{(1)}, x^{(2)}, \dots$ generated by a search heuristic form a homogeneous absorbing Markov chain, then there always exists a potential function such that condition (4) in Theorem 1 holds with equality; namely the function that attributes to each search point the expected optimization time of the algorithm starting in that point.

Lemma 2 [18] *Let $\mathcal{S}$ be a finite search space and $\{x^{(t)}\}_{t \in \mathbb{N}}$ the search points generated by a homogeneous absorbing Markov chain on $\mathcal{S}$. Let T be the random variable that denotes the first point in time $t \in \mathbb{N}$ such that $x^{(t)}$ is optimal.*

Then the drift on the potential function $g: S \rightarrow \mathbb{R}$ with

$$g(x) := \mathbb{E}[T \mid x^{(0)} = x]$$

satisfies

$$\mathbb{E}[g(x^{(t)}) - g(x^{(t+1)}) \mid T > t] = 1.$$

In a way, $\mathbb{E}[T \mid x^{(0)} = x]$ is an “ideal” potential function for Theorem 1. It satisfies the additive drift condition (4) with equality and results in precise upper bound on $\mathbb{E}[T \mid x^{(0)} = x]$. However, the previous theorem is not directly helpful in the run-time analysis of randomized search heuristics. In order to apply the previous theorem, we need to know the exact expected optimization time of an algorithm starting from every point in the search space. But with all this known, Theorem 1 does not provide new information.

Still, the previous theorem indicates that potential functions which approximate the expected optimization time in the respective point are good candidates likely to satisfy the additive drift condition. In the next section, we will see such a potential function suitable for the analysis of the optimization behavior of the (1 + 1) Evolutionary Algorithm on linear functions.

2.2 A Multiplicative Drift Theorem

The drift theorem presented in this subsection can be considered as the multiplicative version of the classical additive result. Since we derive it from the original result, it is clear that the multiplicative version cannot be stronger than the original theorem.

Theorem 3 (Multiplicative Drift) *Let $S \subseteq \mathbb{R}$ be a finite set of positive numbers with minimum $s_{\min}$. Let $\{X^{(t)}\}_{t \in \mathbb{N}}$ be a sequence of random variables over $S \cup \{0\}$. Let T be the random variable that denotes the first point in time $t \in \mathbb{N}$ for which $X^{(t)} = 0$.*

Suppose that there exists a real number $\delta > 0$ such that

$$\mathbb{E}[X^{(t)} - X^{(t+1)} \mid X^{(t)} = s] \geq \delta s \tag{5}$$

holds for all $s \in S$ with $\Pr[X^{(t)} = s] > 0$. Then for all $s_0 \in S$ with $\Pr[X^{(0)} = s_0] > 0$, we have

$$\mathbb{E}[T \mid X^{(0)} = s_0] \leq \frac{1 + \ln(s_0/s_{\min})}{\delta}.$$

Like for the notion of additive drift, we say that the drift of a random process $\{x^{(t)}\}_{t \in \mathbb{N}}$ with respect to a potential function g is *multiplicative* if condition (5) holds for the associated random variables $x^{(t)} := g(x^{(t)})$.

Note that, unlike for Theorem 1, the bound on the expectation of T provided in Theorem 3 depends on the minimal non-zero potential value $s_{\min}$. This dependency is strictly necessary: consider the (deterministic) process given by $X^{(0)} = 1$ and $X^{(t+1)} = \delta X^{(t)}$ for all $t \in \mathbb{N}$. Clearly, this process satisfies condition (5). However, $X^{(t)} > 0$ for all $t \in \mathbb{N}$ and thus T is not even well-defined in this setting.

The advantage of the multiplicative approach is that it allows to use potential functions which are more natural. The most natural potential function, obviously, is the distance of the objective value of the current solution to the optimum. This often is a good choice in the analysis of combinatorial optimization problems. For example, in Sect. 4 we see that the runtimes of the $(1 + 1)$ EA on finding a minimum spanning tree, a shortest path tree, or an Euler tour can be bounded by analyzing this potential function.

Another potential function for which drift analysis has been successfully applied is the *distance in the search space* between the current search points and a (global) optimum.

The typical example for this is the drift analysis for linear functions in Sect. 3, where we use the (weighted) Hamming distance to the optimum as potential for all functions of this class. While being more difficult to analyze, this approach often gives tighter bounds which are independent of the range of the potential fitness values.

Note that multiplicative drift analysis applies to all situations where previously the so-called *method of expected weight decrease* was used. This not rigorously defined method comprises different variants of a proof technique which also builds on the observation that if the drift is multiplicative (that is, condition (5) holds), then at time $t = (1 + \ln(s_0/s_{\min}))/\delta$ the expected potential $X^{(t)}$ is at most s_0/e . Afterwards, various methods (variants of Wald's identity in [11, 19] and Markov's inequality in [2, 23]) are used to show that the expected stopping time $E[T]$ is indeed in this regime. However, the bounds obtained in this way are not the best possible. This is demonstrated in Sect. 3.3 where we replace for the proofs in [19] the method of expected weight decrease by the above multiplicative drift theorem. This results in an immediate improvement of the leading constant in the main runtime bound of [19].

Proof of Theorem 3 Let $g : S \rightarrow \mathbb{R}$ be the function defined by

$$g(s) := 1 + \ln\left(\frac{s}{s_{\min}}\right).$$

Let $R := g(S)$ be the image of g and let $\{Z^{(t)}\}_{t \in \mathbb{N}}$ be the sequence of random variables over $R \cup \{0\}$ given by

$$Z^{(t)} := \begin{cases} 0 & \text{if } X^{(t)} = 0, \\ g(X^{(t)}) & \text{otherwise.} \end{cases}$$

Then T is also the first point in time $t \in \mathbb{N}$ such that $Z^{(t)} = 0$. Suppose $T > t$. Then we have $Z^{(t)} = g(X^{(t)}) > 0$. If $X^{(t+1)} = 0$, then also $Z^{(t+1)} = 0$ and

$$Z^{(t)} - Z^{(t+1)} = 1 + \ln\left(\frac{X^{(t)}}{s_{\min}}\right) \geq 1 = \frac{X^{(t)} - X^{(t+1)}}{X^{(t)}}. \quad (6)$$

Otherwise, $Z^{(t+1)} = g(X^{(t+1)})$ and again

$$Z^{(t)} - Z^{(t+1)} = \ln\left(\frac{X^{(t)}}{X^{(t+1)}}\right) \geq \frac{X^{(t)} - X^{(t+1)}}{X^{(t)}}, \quad (7)$$

where the last inequality follows from

$$\frac{u}{w} = 1 + \frac{u-w}{w} \leq e^{\frac{u-w}{w}}$$

which implies

$$\ln\left(\frac{u}{w}\right) \leq \frac{u-w}{w}$$

and thus

$$\ln\left(\frac{w}{u}\right) \geq \frac{w-u}{w}.$$

for all $u, w \in \mathbb{R}$.

Hence, by (6) and (7), independent of whether $Z^{(t+1)} = 0$ or $Z^{(t+1)} \neq 0$, we have

$$Z^{(t)} - Z^{(t+1)} \geq \frac{X^{(t)} - X^{(t+1)}}{X^{(t)}}.$$

Let $r \in R$. Since g is bijective, there exists a unique $s \in S$ such that $r = g(s)$. Moreover, the events $Z^{(t)} = r$ and $X^{(t)} = s$ coincide. Hence, we have by condition (5) that

$$\mathbb{E}[Z^{(t)} - Z^{(t+1)} \mid Z^{(t)} = r] \geq \frac{\mathbb{E}[X^{(t)} - X^{(t+1)} \mid X^{(t)} = s]}{s} \geq \delta.$$

Finally, we apply Theorem 1 for additive drift and obtain for $s \in S$ with $\Pr[X^{(0)} = s] > 0$ that

$$\mathbb{E}[T \mid X^{(0)} = s] = \mathbb{E}[T \mid Z^{(0)} = g(s)] \leq \frac{g(s)}{\delta} \leq \frac{1 + \ln(s/s_{\min})}{\delta}$$

which concludes the proof of the theorem. $\square$

In Sects. 3 and 4, we demonstrate the strength of this tool by applying it to four well-known problems: the problem of minimizing linear pseudo-Boolean functions, the minimum spanning tree problem, the single-source shortest path problem, and the problem of finding Euler tours.

Recently, Doerr and Goldberg [3] have shown that in Theorem 3, the stopping time T is with high probability at most of the same order as the upper bound on its expectation given in inequality (5), if $X^{(0)}$ is at least $\Omega(n)$. This implies that in the following sections, the implicit upper bound given in Theorem 9 and the bounds in Theorems 15 and 17 also hold with high probability, if we allow a slightly larger leading constant.

3 The Runtime of the (1 + 1) Evolutionary Algorithm on Pseudo-Boolean Functions

Many optimization problems can be phrased as the problem of maximizing or minimizing a pseudo-Boolean function $f: \{0, 1\}^n \rightarrow \mathbb{R}$ where n is a positive integer. In

Algorithm 1: The $(1 + 1)$ Evolutionary Algorithm $((1 + 1)$ EA) with Mutation Rate $1/n$ for Minimizing $f: \{0, 1\}^n \rightarrow \mathbb{R}$

```

1 choose  $x^{(0)} \in \{0, 1\}^n$  uniformly at random;
2 for  $t = 0$  to  $\infty$  do
3   sample  $y^{(t)} \in \{0, 1\}^n$  by flipping each bit in  $x^{(t)}$  with probability  $1/n$ ;
4   if  $f(y^{(t)}) \leq f(x^{(t)})$  then
5      $x^{(t+1)} := y^{(t)}$ 
6   else
7      $x^{(t+1)} := x^{(t)}$ 

```

the setting of randomized search heuristics, such a function f is considered to be a black-box, that is, the optimization process can access f only by evaluating it at limited number of points in $\{0, 1\}^n$.

In this section, we analyze the $(1 + 1)$ Evolutionary Algorithm $((1 + 1)$ EA) for pseudo-Boolean functions (Algorithm 1). This algorithm follows the neighborhood structure imposed by the hypercube on $\{0, 1\}^n$ where two points are adjacent if they differ by exactly one bit, that is, if their Hamming distance is one. The $(1 + 1)$ EA successively attempts to improve the so-far best search point by randomly sampling candidates over $\{0, 1\}^n$ according to probabilities decreasing with the distance to the current optimum.

The *optimization time* of the $(1 + 1)$ EA on a function f is the random variable T that denotes the first point in time $t \in \mathbb{N}$ such that $f(x^{(t)})$ is minimal.

One elementary linear pseudo-Boolean function for which the optimization time $(1 + 1)$ EA has been analyzed (e.g., in [22] and [11]) is the function ONEMAX: $\{0, 1\}^n \rightarrow \mathbb{N}$. This function simply counts the number of one-bits in x , that is,

$$\text{ONEMAX}(x) := |x|_1 = \sum_{i=1}^n x_i. \quad (8)$$

Unlike indicated by the name of this function, we are interested in the time the $(1 + 1)$ EA needs to find its minimum. Thus, in the selection step (Step 4) of each iteration, the $(1 + 1)$ EA accepts the candidate solution $y^{(t)}$ if and only if the number of bits equal to 1 does not increase.

Consider the progress $\Delta^{(t)} := \text{ONEMAX}(x^{(t)}) - \text{ONEMAX}(x^{(t+1)})$ of the $(1 + 1)$ EA in the t th iteration. By construction of the $(1 + 1)$ EA, $\Delta^{(t)}$ cannot be negative. By definition, the number of one-bits $x^{(t)}$ is $\text{ONEMAX}(x^{(t)})$. For each of these one-bits, there is a $(1/n)(1 - 1/n)^{n-1} \geq 1/(en)$ chance that only this one-bit is flipped when sampling $y^{(t)}$, thus increasing the value of $\text{ONEMAX}(x^{(t)})$ by one. Hence,

$$\mathbb{E}[\Delta^{(t)} \mid x^{(t)}] \geq \frac{\text{ONEMAX}(x^{(t)})}{en}.$$

Thus, multiplicative drift analysis (Theorem 3 with $\delta = (en)^2$ and $s_{\min} = 1$) immediately gives us the well-known result

$$\mathbb{E}[T_{\text{ONEMAX}}] \leq en \left(1 + \ln \mathbb{E}[\text{ONEMAX}(x^{(0)})]\right) = en \left(1 + \ln \left(\frac{n}{2}\right)\right).$$

Another elementary linear pseudo-Boolean function is BINVAL . This function maps a bitstring to the binary value it represents (where x_1 represents the lowest and x_n the highest bit).

$$\text{BINVAL}(x) = \sum_{i=1}^n 2^{i-1} x_i. \quad (9)$$

Again, for $\Delta^{(t)} := \text{BINVAL}(x^{(t)}) - \text{BINVAL}(x^{(t+1)})$, we have

$$\mathbb{E}[\Delta^{(t)} \mid x^{(t)}] \geq \frac{\text{BINVAL}(x^{(t)})}{en}$$

and thus

$$\mathbb{E}[T_{\text{BINVAL}}] \leq en \left(1 + \ln \mathbb{E}[\text{BINVAL}(x^{(0)})]\right) = en \left(1 + \ln \left(\frac{2^n - 1}{2}\right)\right).$$

Note, that the previous inequality gives us only a quadratic upper bound of $O(n^2)$ for the expected optimization time of the $(1+1)$ EA on BINVAL . However, it is known that for all linear functions—including BINVAL —the expected optimization time of the $(1+1)$ EA is $O(n \log n)$. We discuss this in the following subsections and give a simplified proof using multiplicative drift analysis.

3.1 Linear Functions

A classical test problem for the runtime analysis of randomized search heuristics is the minimization of *linear functions*.

Let $n \in \mathbb{N}$ be a positive integer. A function $f: \{0, 1\}^n \rightarrow \mathbb{R}$ on n bits is *linear*, if there exists weights $w_1, \dots, w_n \in \mathbb{R}$ such that

$$f(x) = \sum_{i=1}^n w_i x_i$$

for all $x \in \{0, 1\}^n$. In [11] it has been argued and it is easily seen that in the analysis of upper bounds of the expected optimization time of the $(1+1)$ EA on linear functions we may assume without loss of generality that the weights w_i are all positive and sorted, that is,

$$0 < w_1 \leq w_2 \leq \dots \leq w_n. \quad (10)$$

We simply call such weights *monotone*. Moreover, for the runtime bounds we consider in this work it does not matter whether the $(1+1)$ EA minimizes or maximizes

the linear function. This is true since maximizing a function f is equivalent to minimizing $-f$ and vice versa (for $-f$ we again have to invoke above argument which allows us to assume monotonicity of the weights).

Thus, from now on, we suppose that every linear function satisfies condition (10). Furthermore, we formulate all results for the minimization problem, even if the referenced results originally considered the problem of maximizing linear functions.

We have already seen two prominent examples of linear functions, namely the functions ONEMAX and BINVAL. When minimizing ONEMAX, the $(1 + 1)$ EA accepts a new bit string in the selection step (Step 4) if the number of one-bits did not increase. In contrast, when minimizing BINVAL, the inequality $2^k > \sum_{i=1}^{k-1} 2^{i-1}$ implies that the $(1 + 1)$ EA accepts a new bit string if and only if the highest-index bit that is touched in the mutation step (Step 3) is flipped from one to zero.

In spite of this difference in behavior, Droste, Jansen, and Wegener showed in their seminal paper [11] that for all linear functions the expected optimization time of the $(1 + 1)$ EA is $\Theta(n \log n)$.

Theorem 4 [11] *For all positive integers $n \in \mathbb{N}$, the expected running time of the $(1 + 1)$ EA on the class of linear functions with non-zero weights is $\Theta(n \log n)$.*

The proof of Droste, Jansen, and Wegener applies a level based argument to the potential function (called *artificial fitness function*) $g: \{0, 1\}^n \rightarrow \mathbb{R}$ such that for all $x \in \{0, 1\}^n$

$$g(x) = \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} x_i + \sum_{i=\lfloor \frac{n}{2} \rfloor + 1}^n 2x_i. \quad (11)$$

A much easier proof avoiding partitioning arguments and instead working completely in the framework of drift analysis, was given by He and Yao in [18]. There, additive drift analysis is applied to the potential function $h: \{0, 1\}^n \rightarrow \mathbb{R}$ such that

$$h(x) = \ln \left(1 + \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} x_i + \sum_{i=\lfloor \frac{n}{2} \rfloor + 1}^n cx_i \right)$$

for all $x \in \{0, 1\}^n$ already introduced as (1) in the introduction. For this function, with $1 < c \leq 2$ chosen arbitrarily, they show that for all $x \in \{0, 1\}^n \setminus \{(0, \dots, 0)\}$

$$\mathbb{E}[h(x^{(t)}) - h(x^{(t+1)}) \mid x^{(t)} = x] = \Omega(1/n).$$

Afterwards, they apply Theorem 1 to show Theorem 4. However, while this approach strongly reduced the complexity of the proof in [11], introducing the natural logarithm into the potential function still resulted in unnecessary case distinctions and even inconsistencies in an early version of the proof [16, 17].

3.2 The Drift for Linear Functions is Multiplicative

In this subsection, we give a simple proof of the fact that the $(1 + 1)$ EA optimizes any linear function in expected time $O(n \log n)$. Our proof uses the Multiplicative

Drift Theorem (Theorem 3). Although proofs for Theorem 4 are known [11, 18, 19], we present this alternative approach to demonstrate the strength of the multiplicative version of the classical drift theorem.

In order to apply Theorem 3 we need a suitable potential function. For this, we choose the function $g: \{0, 1\}^n \rightarrow \mathbb{R}$ such that²

$$g(x) = \sum_{i=1}^n \left(1 + \frac{i}{n}\right) x_i$$

for all $x \in \{0, 1\}^n$. This function defines the potential as the *weighted* distance of the current search point to the optimum (the all-zero string) in the search space. More precisely, it counts the number of one-bits, where each bit is assigned a weight between one and two, such that bits which have higher weight in the objective function f also have higher weight in g .

We show that the drift of the $(1 + 1)$ EA with respect to g is multiplicative, that is, that condition (5) holds.

Lemma 5 *Let $n \in \mathbb{N}$ be a positive integer. Let $f: \{0, 1\}^n \rightarrow \mathbb{R}$ be a linear function with monotone weights and let $g: \{0, 1\}^n \rightarrow \mathbb{R}$ be the linear potential function satisfying $g(x) = \sum_{i=1}^n g_i x_i$ for all $x \in \{0, 1\}^n$, where $g_i = 1 + i/n$ for all $i \in \{1, \dots, n\}$.*

Let $x \in \{0, 1\}^n$ and let $y \in \{0, 1\}^n$ be randomly chosen by flipping each bit in x with probability $1/n$. Let $\Delta(x) := g(x) - g(y)$ if $f(y) \leq f(x)$ and $\Delta(x) = 0$ otherwise. Then

$$\mathbb{E}[\Delta(x)] \geq \frac{g(x)}{4en}.$$

This lemma implies that *at every point in the search space* the drift is at least linear in the current potential value. Thus, the multiplicative drift condition (5) holds and Theorem 4 follows directly by applying Theorem 3.

Proof of Lemma 5 Since $\mathbb{E}[\Delta(x) \mid f(y) > f(x)] = 0$, we have by the law of total expectation that

$$\mathbb{E}[\Delta(x)] = \mathbb{E}[g(x) - g(y) \mid f(y) \leq f(x)] \Pr[f(y) \leq f(x)]. \quad (12)$$

Let $I = \{i \in \{1, \dots, n\} : x_i = 1\}$. We may distinguish three events (cases).

- (C₁) There is no index $i \in I$ such that $y_i = 0$ and $f(y) \leq f(x)$ holds, that is, $x = y$.
- (C₂) There is exactly one index $i \in I$ such that $y_i = 0$ and $f(y) \leq f(x)$ holds.
- (C₃) There are at least two different indices $j, \ell \in I$ such that $y_j = 0$ and $y_\ell = 0$ and $f(y) \leq f(x)$ holds.

²We might as well perform our analysis of g as defined in (11). However, our choice of g does not make the somewhat artificial binary distinction between bits with high and low indices and, thus, seems to be more natural.

The only possibility for the event (C_1) to hold is if $x = y$. Therefore,

$$\mathbb{E}[g(x) - g(y) \mid (C_1)] = 0. \quad (13)$$

Next, suppose the event (C_3) holds. By linearity of expectation, we have

$$\mathbb{E}[g(x) - g(y) \mid (C_3)] = \sum_{i=1}^n \mathbb{E}[g_i x_i - g_i y_i \mid (C_3)] = \sum_{i=1}^n g_i \mathbb{E}[x_i - y_i \mid (C_3)].$$

On the one hand, the event (C_3) implies that there are (at least) two indices j and ℓ in $\{1, \dots, n\}$ for which $x_j = x_\ell = 1$ and $y_j = y_\ell = 0$. Since $g_j \geq 1$ and $g_\ell \geq 1$, we have

$$\sum_{i \in I} g_i \mathbb{E}[x_i - y_i \mid (C_3)] \geq 2.$$

On the other hand, if $i \in \{1, \dots, n\} \setminus I$ then

$$g_i \mathbb{E}[x_i - y_i \mid (C_3)] = -g_i \Pr[y_i = 1 \mid (C_3)] \geq -\frac{g_i}{n},$$

since the condition (C_3) does not increase the probability of $1/n$ that the $y_i = 0$. Therefore, since the g_i 's are at most two, we have

$$\mathbb{E}[g(x) - g(y) \mid (C_3)] \geq 2 - \frac{1}{n} \sum_{i \notin I} g_i \geq 0. \quad (14)$$

Therefore, by the law of total expectation and by (12), (13) and (14), we have

$$\mathbb{E}[\Delta(x)] \geq \mathbb{E}[g(x) - g(y) \mid (C_2)] \Pr[(C_2)] \quad (15)$$

and can focus on the event (C_2) .

Suppose that (C_2) holds. For every $i \in I$, we distinguish two events:

- (A_i) The i th bit is the only one-bit in x that flips, none of the zero-bits at the positions larger than i flips, and $f(y) \leq f(x)$ holds.
- (B_i) The i th bit is the only one-bit in x that flips, at least one of the zero-bits at the positions larger than i that flips, and $f(y) \leq f(x)$ holds.

We substitute the right side in (15) and obtain

$$\mathbb{E}[\Delta(x)] \geq \sum_{i \in I} \mathbb{E}[\Delta(x) \mid (A_i)] \Pr[(A_i)] + \mathbb{E}[\Delta(x) \mid (B_i)] \Pr[(B_i)]. \quad (16)$$

Let $i \in I$ and suppose that the condition (A_i) holds. Then we have $y_i = 0$ and $y_j = x_j$ for all $j > i$. For a lower bound on $\mathbb{E}[\Delta(x) \mid (A_i)]$, we may suppose that $x_j = 0$ for all $j < i$ and that every flip of a bit with index $j < i$ is accepted. Therefore, since $i \leq n$

$$\mathbb{E}[\Delta(x) \mid (A_i)] \geq 1 + \frac{i}{n} - \sum_{j=1}^{i-1} \frac{1}{n} \left(1 + \frac{j}{n}\right) = 1 + \frac{1}{n} - \frac{i(i-1)}{2n^2} \geq 1 - \frac{i-3}{2n}.$$

and thus $E[\Delta(x) \mid (A_i)]$ is positive. Furthermore, $\Pr[(A_i)] \geq \frac{1}{n}(1 - \frac{1}{n})^{n-1}$ which is the probability that only the i th bit flips. Hence,

$$E[\Delta(x) \mid (A_i)] \Pr[(A_i)] \geq \frac{1}{n} \left(1 - \frac{1}{n}\right)^{n-1} \left(1 - \frac{i-3}{2n}\right). \quad (17)$$

Next, suppose that condition (B_i) holds. Then we have $y_i = 0$ and $y_\ell = 1$ for all $\ell \in I \setminus \{i\}$, and there exists a $j > i$ with $j \notin I$ such that $y_j = 1$. In order to satisfy $f(y) \leq f(x)$, $w_j = w_i$ has to hold. This implies $x_\ell = y_\ell$ for all $\ell \in \{1, \dots, n\} \setminus \{i, j\}$. To see this, recall that the w_i 's are monotone and we condition on the event that the i th bit is the only bit that flips from one to zero.

Let $J(i) = \{j \in \{i+1, \dots, n\} : x_j = 0 \text{ and } w_j = w_i\}$. For $j \in J(i)$ let $B_{i,j}$ be the event that $y_i = 0$, $y_j = 1$, and $y_\ell = x_\ell$ for ℓ not i or j . Then

$$E[\Delta(x) \mid (B_i)] \Pr[(B_i)] = \sum_{j \in J(i)} E[\Delta(x) \mid B_{i,j}] \Pr[B_{i,j}].$$

We substitute $E[\Delta(x) \mid B_{i,j}] = -\frac{j-i}{n}$ and $\Pr[B_{i,j}] = \frac{1}{n^2}(1 - \frac{1}{n})^{n-2}$ in the previous equation. Since these conditional expectations are always negative, we may pessimistically assume that $J(i) = \{i+1, \dots, n\}$ and get

$$\sum_{j \in J(i)} E[\Delta(x) \mid B_{i,j}] = -\frac{(n-i)}{n} \frac{(n+1-i)}{2} \geq -\left(1 - \frac{1}{n}\right) \frac{(n+1-i)}{2}$$

and therefore

$$E[\Delta(x) \mid (B_i)] \Pr[(B_i)] \geq -\frac{1}{n} \left(1 - \frac{1}{n}\right)^{n-1} \frac{n+1-i}{2n}. \quad (18)$$

Finally, we substitute (17) and (18) in (16) and derive

$$E[\Delta(x)] \geq \frac{1}{n} \left(1 - \frac{1}{n}\right)^{n-1} \sum_{i \in I} 1 - \frac{i-3}{2n} - \frac{n+1-i}{2n} = \frac{1}{n} \left(1 - \frac{1}{n}\right)^{n-1} \frac{n+2}{4n} \sum_{i \in I} 2.$$

Since $g_i = 1 + i/n \leq 2$ for all $i \in I$, we have $\sum_{i \in I} 2 \geq g(x)$ and therefore

$$E[\Delta(x)] \geq \frac{g(x)}{4en}$$

which concludes the proof of the lemma. $\square$

3.3 Distribution-Based Versus Point-Wise Drift

In this subsection we show an almost tight upper bound on the expected optimization time of the $(1+1)$ EA on linear functions.

If we take a closer look at Lemma 5, we see that it holds *point-wise*, that is, it guarantees

$$E[g(x^{(t)}) - g(x^{(t+1)}) \mid x^{(t)} = x] \geq \frac{g(x)}{4en} \quad (19)$$

for all $x \in \{0, 1\}^n \setminus \{(0, \dots, 0)\}$. This condition is far stronger than the positive *average* drift condition (5) (that is, the former implies the latter) which only requires

$$\mathbb{E}[g(x^{(t)}) - g(x^{(t+1)}) \mid g(x^{(t)}) = s] \geq \frac{s}{4en} \quad (20)$$

for all $s \in \mathbb{R}$ such that $\Pr[g(x^{(t)}) = s] > 0$.

The advantage of the stronger point-wise drift assumption is that it immediately guarantees that the result of Theorem 4 holds for all initial individuals.

The main reason, however, for not using the weaker condition (20) is that this requires a deeper understanding of the probability distribution of $x^{(t)}$.

Let us stress that finding a potential function satisfying the stronger point-wise drift condition is usually very tricky. For example, one may ask why not take $\text{ONEMAX}(x)$ as potential function to bound the expected optimization time of the $(1 + 1)$ EA for minimizing linear functions.

However, an easy observation reveals that there is an objective function f and a search point x such that g yields too small a drift with respect to f . To see this, let $x = (x_1, \dots, x_n) := (0, \dots, 0, 1)$ and let $f := \text{BINVAL}$ be the function to be minimized. Then the point-wise drift (19) with respect to ONEMAX is only $1/n^2$. This example shows that finding a potential function yielding point-wise drift for all x and all f may be difficult. This observation is not to be confused with that in the discussion following (9). There, we determined the drift using the function BINVAL itself as potential. Here, we use ONEMAX , that is, the 1-norm as potential function.

Jägersküpfer [19] was the first to overcome the difficulties of point-wise drift. While he still avoids completely analyzing the actual distribution of $x^{(t)}$, he does show the following property of this distribution which in turn allows him to use an average drift approach. In this way, he omits the need for point-wise drift. Jägersküpfer's simple observation is that at any time step t , the more valuable bits are more likely to be in the right setting (cf. Theorem 1 in [19]).

Theorem 6 [19] *Let $n \in \mathbb{N}$ be a positive integer and let $x^{(t)}$ denote the random individual (distributed over $\{0, 1\}^n$) after $t \in \mathbb{N}$ iterations of the $(1 + 1)$ EA minimizing a linear function $f: \{0, 1\}^n \rightarrow \mathbb{R}$. Then*

$$\Pr[x_1^{(t)} = 0] \leq \dots \leq \Pr[x_n^{(t)} = 0].$$

Moreover, for all $k \in \{0, \dots, n\}$, this statement remains true if we condition on $\text{ONEMAX}(x) = k$.

Using this theorem, he was able to show a lower bound of $\Omega(1/n)$ for the drift of ONEMAX as potential function for any linear function.

Lemma 7 [19] *Let $n \in \mathbb{N}$ be a positive integer and let $f: \{0, 1\}^n \rightarrow \mathbb{N}$ be a linear function. Let $x^{(t)}$ be the individual in the t th iteration of the $(1 + 1)$ EA minimizing f . Then*

$$\mathbb{E}[\text{ONEMAX}(x^{(t)}) - \text{ONEMAX}(x^{(t+1)}) \mid \text{ONEMAX}(x^{(t)}) = k] \geq \frac{(e-2)k}{en}.$$

holds for all $k \in \{0, \dots, n\}$ and $t \in \mathbb{N}$.

In addition to a more natural proof of the $O(n \log n)$ bound for expected optimization time of the $(1 + 1)$ EA minimizing a linear function, Jägersküpfer was able to give a meaningful upper bound on the leading constant (cf. [19, Theorem 2]).

Theorem 8 [19] *For all positive integers $n \in \mathbb{N}$, the expected optimization time of the $(1 + 1)$ EA minimizing a linear function on n bits is at most of order $(1 + o(1))2.02en \ln n$.*

Using multiplicative drift analysis (Theorem 3 with parameters $\delta = \frac{e-2}{en}$, $s_{\min} = 1$, and $x^{(0)} \leq n$) on the result of Lemma 7 and thus replacing the halving argument employed by Jägersküpfer for the proof of Theorem 8, the constant of $2.02e$ in the upper bound of the previous theorem instantly improves to $1.39e$. In the light of our lower bound of $1.00e$, to be proven in the next subsection, this is a considerable progress.

Theorem 9 *For all positive integers $n \in \mathbb{N}$, the expected optimization time of the $(1 + 1)$ EA minimizing a linear function on n bits is at most of order $(1 + o(1))\frac{e}{e-2}n \ln n \approx (1 + o(1))1.39en \ln n$.*

3.4 The $(1 + 1)$ EA Optimizes OneMax Faster than Any Other Function with a Unique Global Optimum

In this section, we show that the expected optimization time of $(1 + 1)$ EA on any pseudo-Boolean function with a unique global optimum is at least as large as its expected optimization time on the basic function ONEMAX. In particular, this is true for every linear function with non-zero coefficients.

In other words, if a function is easier to optimize than ONEMAX, then this can only be due to the fact that it has more than one global optimum. The general lower bound then follows from the following theorem by Doerr, Fouz and Witt [7], which provides a lower bound for ONEMAX.

Theorem 10 [7] *For all positive integers $n \in \mathbb{N}$, the expected optimization time of the $(1 + 1)$ EA minimizing ONEMAX on n bits is at least $(1 - o(1))en \ln n$.*

Thus, it remains to show that ONEMAX is optimized fastest. The result itself was announced by Scheder and Welzl [26]. Their idea to prove this statement, however, differs from the one given below.

Theorem 11 *Let $n \in \mathbb{N}$ be a positive integer. The expected optimization time of the $(1 + 1)$ EA on any function $f: \{0, 1\}^n \rightarrow \mathbb{R}$ that has a unique global optimum is at least as large as its expected optimization time on ONEMAX.*

Theorems 10 and 11 immediately yield the following.

Corollary 12 *For all positive integers $n \in \mathbb{N}$, the expected optimization time of the $(1 + 1)$ EA minimizing a function with a unique global optimum on n bits is at least $(1 - o(1))en \ln n$.*

For the proof of Theorem 11 we first show a preliminary lemma. It formalizes the following intuition. Let x and $\tilde{x}$ be two search points such that $|x|_1 \leq |\tilde{x}|_1$. Then the probability that the $(1 + 1)$ EA samples a new search point with exactly $j < |x|_1$ one-bits from x is at least as big as sampling it from $\tilde{x}$.

Lemma 13 *Let $n \in \mathbb{N}$ with $n \geq 1$. Let $x, \tilde{x} \in \{0, 1\}^n$ with $|x|_1 \leq |\tilde{x}|_1$. Let y and $\tilde{y}$ be two random points in $\{0, 1\}^n$ obtained from x and $\tilde{x}$ by independently flipping with probability $1/n$ each bit of x and $\tilde{x}$, respectively.*

Then for every $j \in \{0, \dots, |x|_1 - 1\}$,

$$\Pr[|y|_1 = j] \geq \Pr[|\tilde{y}|_1 = j].$$

Proof Let $k := |x|_1$. The lemma holds trivially if $|\tilde{x}|_1 = k$.

Suppose that $|\tilde{x}|_1 = k + 1$. Then

$$\Pr[|y|_1 = j] = \sum_{i=0}^{\min\{j, n-k\}} \binom{k}{j-i} \binom{n-k}{i} \left(\frac{1/n}{1-1/n}\right)^{k-j+2i} (1-1/n)^n$$

and

$$\Pr[|\tilde{y}|_1 = j] = \sum_{i=0}^{\min\{j, n-k-1\}} \binom{k+1}{j-i} \binom{n-k-1}{i} \left(\frac{1/n}{1-1/n}\right)^{k+1-j+2i} (1-1/n)^n.$$

As all summands in the previous two equations are positive, it suffices to see that the quotient

$$\frac{\binom{k}{j-i} \binom{n-k}{i} \left(\frac{1/n}{1-1/n}\right)^{k-j+2i}}{\binom{k+1}{j-i} \binom{n-k-1}{i} \left(\frac{1/n}{1-1/n}\right)^{k+1-j+2i}} = \frac{(k+1-j+i)(n-k)(n-1)}{(k+1)(n-k-i)}$$

is minimal for $i = 0$ and $j = k - 1$ and therefore at least 1 for all values $0 \leq i \leq \min\{j, n - k - 1\}$.

Thus, for $|\tilde{x}|_1 = k + 1$ the lemma also holds. Finally, for $|\tilde{x}|_1 > k + 1$, the lemma follows by induction based on the case $|\tilde{x}|_1 = k + 1$. $\square$

To prove the main result of this section, Theorem 11, we need some additional notation. Let f be a function with a unique global optimum x^* . Without loss of generality, we may assume that $x^* := (0, \dots, 0)$ is the unique minimum of f . This is justified by the observation that the $(1 + 1)$ EA treats the bit-values 0 and 1 symmetrically, that is, we might reinterpret one-bits in x^* as zero-bits without changing the behavior of the algorithm.

Let $\mu(x) := \mathbb{E}[T_{\text{ONEMAX}} \mid x^{(0)} = x]$ and $\tilde{\mu}(x) := \mathbb{E}[T_f \mid x^{(0)} = x]$ be the expected optimization times of the $(1 + 1)$ EA starting in the point x and minimizing ONEMAX and f , respectively.

For every $k \in \{0, \dots, n\}$ let

$$\mu_k := \min\{\mu(x) \mid x \in \{0, 1\}^n, |x|_1 = k\}$$

be the optimization time of the $(1 + 1)$ EA optimizing ONEMAX starting in a point with exactly k one-bits. Note that, due to the symmetry of the function ONEMAX, $\mu_k = \mu(x)$ for every $x \in \{0, 1\}^n$ with exactly k one-bits.

For every $k \in \{0, \dots, n\}$, let

$$\tilde{\mu}_k := \min\{\tilde{\mu}(x) \mid x \in \{0, 1\}^n, |x|_1 \geq k\}$$

be the minimum optimization time of the $(1 + 1)$ EA minimizing f and starting in a point x with *at least* k one-bits. Note the difference to the definition of μ_k . Since the minimum in the definition of the $\tilde{\mu}_k$'s is taken over a set which shrinks with growing k , we have

$$\tilde{\mu}_0 \leq \dots \leq \tilde{\mu}_n.$$

Proof of Theorem 11 We inductively show for all k that $\mu_k \leq \tilde{\mu}_k$. Clearly $\mu_0 = 0 = \tilde{\mu}_0$. Therefore, let $k \in \{0, \dots, n-1\}$ and suppose that $\mu_i \leq \tilde{\mu}_i$ for all $i \leq k$.

Let $x \in \{0, 1\}^n$ with $|x|_1 = k+1$ be arbitrary and let $y \in \{0, 1\}^n$ be a random point generated by flipping each bit in x independently with probability $1/n$.

The $(1 + 1)$ EA minimizing ONEMAX and starting in x accepts y in the selection step (Step 4) if and only if $|y|_1 \leq |x|_1$. Furthermore, we have $\mu(x) = \mu(y)$ if $|y|_1 = |x|_1 = k+1$. Thus,

$$\mu(x) = 1 + \mu(x) \Pr[|y|_1 \geq k+1] + \sum_{j=0}^k \mathbb{E}[\mu(y) \mid |y|_1 = j] \Pr[|y|_1 = j]$$

and therefore

$$\mu_{k+1} = 1 + \mu_{k+1} \Pr[|y|_1 \geq k+1] + \sum_{j=0}^k \mu_j \Pr[|y|_1 = j]. \quad (21)$$

Next, let $\tilde{x} \in \{0, 1\}^n$ be chosen arbitrarily such that $|\tilde{x}|_1 \geq k+1$ and $\tilde{\mu}_{k+1} = \tilde{\mu}(\tilde{x})$. Furthermore, let $\tilde{y} \in \{0, 1\}^n$ be a random point generated by flipping each bit in $\tilde{x}$ independently with probability $1/n$. Let $\tilde{z} = \tilde{y}$ if $f(\tilde{y}) \leq f(\tilde{x})$ and $\tilde{z} = \tilde{x}$ otherwise.

Then

$$\begin{aligned} \tilde{\mu}_{k+1} &= \tilde{\mu}(\tilde{x}) = 1 + \mathbb{E}[\tilde{\mu}(\tilde{z}) \mid |\tilde{z}|_1 \geq k+1] \Pr[|\tilde{z}|_1 \geq k+1] \\ &\quad + \sum_{j=0}^k \mathbb{E}[\tilde{\mu}(\tilde{z}) \mid |\tilde{z}|_1 = j] \Pr[|\tilde{z}|_1 = j] \end{aligned}$$

and therefore, by definition of $\tilde{\mu}_j$ and in particular their monotonicity,

$$\tilde{\mu}_{k+1} \geq 1 + \tilde{\mu}_{k+1} \Pr[|\tilde{z}|_1 \geq k+1] + \sum_{j=0}^k \tilde{\mu}_j \Pr[|\tilde{z}|_1 = j].$$

We rewrite this inequality and get

$$\begin{aligned} \tilde{\mu}_{k+1} &\geq 1 + \tilde{\mu}_{k+1} \Pr[|\tilde{z}|_1 \geq k+1] + \sum_{j=0}^k \tilde{\mu}_j \Pr[|y|_1 = j] \\ &\quad - \sum_{j=0}^k \tilde{\mu}_j (\Pr[|y|_1 = j] - \Pr[|\tilde{z}|_1 = j]). \end{aligned} \quad (22)$$

Now, for all $0 \leq j \leq k$, we have

$$\Pr[|\tilde{z}|_1 = j] \leq \Pr[|\tilde{y}|_1 = j] \leq \Pr[|y|_1 = j]. \quad (23)$$

The first inequality in (23) holds, since we have $|\tilde{x}|_1 > j$ and therefore the event $|\tilde{z}|_1 = j$ implies that $\tilde{z} = \tilde{y}$ and thus that $|\tilde{y}|_1 = j$. The second inequality in (23) follows from Lemma 13, since $|x|_1 = k+1 \leq |\tilde{x}|_1$.

As discussed above, the quantities $\tilde{\mu}_i$ increase monotonically in i . Therefore, since (23) guarantees that all terms in the second sum in (22) are non-negative, we have

$$\begin{aligned} \tilde{\mu}_{k+1} &\geq 1 + \tilde{\mu}_{k+1} \Pr[|\tilde{z}|_1 \geq k+1] + \sum_{j=0}^k \tilde{\mu}_j \Pr[|y|_1 = j] \\ &\quad - \tilde{\mu}_{k+1} \sum_{j=0}^k (\Pr[|y|_1 = j] - \Pr[|\tilde{z}|_1 = j]). \end{aligned}$$

Substituting

$$\Pr[|\tilde{z}|_1 \geq k+1] - \sum_{j=0}^k \Pr[|y|_1 = j] + \sum_{j=0}^k \Pr[|\tilde{z}|_1 = j] = \Pr[|y|_1 \geq k+1]$$

in the previous inequality then gives us

$$\tilde{\mu}_{k+1} \geq 1 + \tilde{\mu}_{k+1} \Pr[|y|_1 \geq k+1] + \sum_{j=0}^k \tilde{\mu}_j \Pr[|y|_1 = j].$$

Therefore, the induction hypothesis yields that

$$\tilde{\mu}_{k+1} \geq 1 + \tilde{\mu}_{k+1} \Pr[|y|_1 \geq k+1] + \sum_{j=0}^k \mu_j \Pr[|y|_1 = j].$$

We subtract both sides of (21) from the previous inequality and immediately get $\tilde{\mu}_{k+1} \geq \mu_{k+1}$ which concludes the induction.

Thus, for all $x \in \{0, 1\}^n$, we have

$$\mu(x) = \mu_{|x|_1} \leq \tilde{\mu}_{|x|_1} \leq \tilde{\mu}(x).$$

Consequently, $E[T_{\text{ONEMAX}}] \leq E[T_f]$ holds. $\square$

4 Multiplicative Drift on Combinatorial Problems

So far, we have seen that multiplicative drift analysis can be used to simplify the run-time analysis of the $(1 + 1)$ EA on linear pseudo-Boolean functions while producing sharper bounds. In this section, we see that optimization processes with multiplicative drift occur quite naturally in combinatorial optimization too. We demonstrate this claim on two prominent examples, the minimum spanning tree problem and the single source shortest path problem.

4.1 The Minimum Spanning Tree Problem

In this subsection, we consider the minimum spanning tree (MST) problem analyzed in [23]. Let $G = (V, E)$ be a connected graph with n vertices, m edges $e_1, \dots, e_m$, and positive integer edge weights $w_1, \dots, w_m$. In [23], a spanning tree is represented by a bit string $x \in \{0, 1\}^m$ with $x_i = 1$ marking the presence of the edge e_i in the tree.³

The fitness value of such a tree is defined by $w(x) = \sum_{i=1}^m w_i x_i + p(x)$, with $p(x)$ being a penalty term ensuring that once the $(1 + 1)$ EA has found a spanning tree it does no longer accept bit strings that do not represent spanning trees (a new bit-strings is accepted if the fitness value decreases). The minimum weight of a spanning tree is denoted by w_{opt} and the maximal edge weight by w_{max} .

In Lemma 1 of [23], Neumann and Wegener derive from [21] the following statement.

Lemma 14 [23] *Let $x \in \{0, 1\}^m$ be a search point describing a non-minimum spanning tree. Then there exist a $k \in \{1, \dots, n - 1\}$ and k different accepted 2-bit flips such that the average weight decrease of these flips is at least $(w(x) - w_{\text{opt}})/k$.*

Multiplicative drift analysis now gives us a reasonably small constant in the upper bound of the expected optimization time of the $(1 + 1)$ EA on the MST problem.

Theorem 15 *The expected optimization time of the $(1 + 1)$ EA on the MST problem starting with an arbitrary spanning tree of a non-empty graph is at most $2em^2(1 + \ln m + \ln w_{\text{max}})$.*

Proof For all $t \in \mathbb{N}$, let $x^{(t)}$ be the search point of the $(1 + 1)$ EA for the MST problem at time t and let $X^{(t)} = w(x^{(t)}) - w_{\text{opt}}$. Then

$$X^{(t)} - X^{(t+1)} = w(x^{(t)}) - w(x^{(t+1)}).$$

Now, let $t \in \mathbb{N}$ and $x \in \{0, 1\}^n \setminus \{(0, \dots, 0)\}$ be fixed. Let the points $y_{(1)}, \dots, y_{(k)}$ with $k \in \{0, \dots, n - 1\}$ be the k distinct search points in $\{0, 1\}^m$ generated from x by

³In [23], Neumann and Wegener actually analyze the MST problem on the larger search space consisting of all subgraphs of G . In this section, we restrict the problem to the sub-problem of finding a MST when starting with an arbitrary spanning tree. It is shown in [23] that this sub-problem is the bottleneck in the runtime analysis of the $(1 + 1)$ EA optimizing the MST problem.

the k different 2-bit flips according to Lemma 14. That is, we have $w(y_{(i)}) \leq w(x)$ for all $i \in \{1, \dots, k\}$ and

$$\sum_{i=1}^k (f(x) - f(y_{(i)})) \geq w(x) - w_{\text{opt}}. \quad (24)$$

Since the $y_{(i)}$'s are each generated from x by a 2-bit flip, we have

$$\Pr[x^{(t+1)} = y_{(i)} \mid x^{(t)} = x] = \left(1 - \frac{1}{m}\right)^{m-2} \left(\frac{1}{m}\right)^2 \quad (25)$$

for all $i \in \{1, \dots, k\}$. Furthermore

$$\mathbb{E}[X^{(t)} - X^{(t+1)} \mid x^{(t)} = x, x^{(t+1)} = y_{(i)}] = w(x) - w(y_{(i)}) \quad (26)$$

holds for all $i \in \{1, \dots, k\}$.

The $(1 + 1)$ EA never increases the current w -value of a search point, that is, $X^{(t)} - X^{(t+1)}$ is non-negative. Thus, we have by (25) and (26) that

$$\mathbb{E}[X^{(t)} - X^{(t+1)} \mid x^{(t)} = x] \geq \sum_{i=1}^k (w(x) - w(y_{(i)})) \left(1 - \frac{1}{m}\right)^{m-2} \left(\frac{1}{m}\right)^2$$

and therefore, by inequality (24), we have for all $x \in \{0, 1\}^m$ that

$$\mathbb{E}[X^{(t)} - X^{(t+1)} \mid x^{(t)} = x] \geq \frac{w(x) - w_{\text{opt}}}{em^2}.$$

In other words,

$$\mathbb{E}[X^{(t)} - X^{(t+1)} \mid X^{(t)}] \geq \frac{X^{(t)}}{em^2}$$

and since $1 \leq X^{(t)} \leq mw_{\text{max}}$ the theorem follows from Theorem 3 with parameters $\delta = \frac{1}{em^2}$, $s_{\text{min}} = 1$, and $x^{(0)} \leq mw_{\text{max}}$. $\square$

4.2 The Single-Source Shortest Path Problem

In [2], Baswana, Biswas, Doerr, Friedrich, Kurur, and Neumann study an evolutionary algorithm that solves the single-source shortest path (SSSP) problem on a directed graph with n vertices via evolving a shortest-path tree. In the analysis of the upper bound for the expected optimization time, the authors introduce the *gap* g_i in iteration i as the difference in fitness between the current shortest-path tree candidate and an optimal shortest-path tree.

For every vertex in the tree, its *weight* in the tree is defined as the sum over the weights of edges (which are integers in the set $\{1, \dots, w_{\text{max}}\}$) in the paths leading to the root vertex, or as the penalty term nw_{max} if the vertex is not connected to the root. The fitness of a shortest-path tree candidate is then the sum over the weights of all vertices in the tree. Thus the maximal gap is n^2w_{max} . In Lemma 1 of [2], the authors then provide the following statement.

Lemma 16 [2] *Let g_i denote the gap after i mutations. Then it holds for the conditional expectation $E[g_{i+1} \mid g_i = g]$ that*

$$E[g_{i+1} \mid g_i = g] \leq g \left(1 - \frac{1}{3 \cdot n^3} \right).$$

To this, we can directly apply Theorem 3 with parameters $\delta = \frac{1}{3 \cdot n^3}$, $s_{\min} = 1$, and $x^{(0)} \leq n^2 w_{\max}$ taking the gap as a potential. We obtain the following result with a precise constant for the upper bound.

Theorem 17 *The expected optimization time of the $(1 + 1)$ EA in [2] on the SSSP problem starting with an arbitrary shortest-path tree candidate is at most $3n^3(1 + 2 \ln n + \ln w_{\max})$.*

4.3 The Euler Tour Problem

The Euler tour problem is to find a Euler tour (a closed walk that visits every edge exactly once) in an input graph which permits such a tour.

In [5], possible variants of the $(1 + 1)$ EA for the Euler tour problem are analyzed. For the variant using the so-called edge-based distribution on cycle covers, the search space consists of all ways to cover the input graph with edge-disjoint cycles. In other words, a search point (*cycle cover*) is a collection of cycles in the graph such that each edge appears in exactly one cycle.

The definition of a basic mutation is rather involved since it is based on pairing elements in the graphs adjacency list (see [5] for the precise definition). For our purposes, it is sufficient to understand the effect of a *basic mutation*. In a basic mutation, we first choose a random edge of the graph. We then choose one of its end-vertices at random. Finally, we choose a second random edge at this end-vertex. If both edges are in different cycles, the basic mutation operator joins these cycles, reducing the size of the collection by one. A general mutation consists of $\text{Pois}(1) + 1$ applications of the basic mutation operator.

The fitness of a cycle cover is given by the total number of cycles in the collection. Thus, a fitness of one implies that the graph is covered by a single cycle—an Euler tour.

Finding such a tour is then a minimization problem over this search space. For this setup, the following statement is implicitly shown in the proof of Theorem 3.

Lemma 18 *In a single iteration of the $(1 + 1)$ EA in [5] for the Euler tour problem using the edge-based distribution and starting with an arbitrary cycle cover, the probability to decrease the fitness $f(x)$ of the current search point x by one (provided it was not minimal before) is at least $f(x)/em$ where m is the number of edges of the input graph.*

If we set the fitness minus one as potential, this lemma immediately implies that the drift is at least $\frac{f(x)}{em}$. Moreover, the starting potential is at most $m/3$ (each tour has at least three edges). Again, we can apply Theorem 3 with parameters $\delta = \frac{1}{em}$,

$s_{\min} = 1$ and $x_0 \leq m/3$ and reproduce the upper bound the expected optimization time, specifying the leading constant in the process.

Theorem 19 *The expected optimization time of the $(1 + 1)$ EA in [5] for the Euler tour problem using the edge-based distribution and starting with an arbitrary cycle cover is at most $em \ln m$, where m is the number of edges in the input graph.*

5 Discussion and Outlook

In this work, we showed that the multiplicative drift condition (5) occurs naturally in the runtime analysis of the $(1 + 1)$ EA for a number of prominent optimization problems (linear functions, minimum spanning trees, shortest paths, and Euler tours). In such situations our multiplicative drift theorem (Theorem 5) yields good runtime bounds.

We applied this tool to various settings. First, we used it to gain new insight in the classical problem of how the $(1 + 1)$ EA optimizes linear functions.

We presented a simplified proof of the, by now, well-known fact that the $(1 + 1)$ EA with mutation probability $1/n$ optimizes any linear function in time $O(n \log n)$. Moreover, we applied our result to the distribution-based drift analysis of Jägersküpfer and obtained a new upper bound of $(1 + o(1))1.39en \ln n$ for the expected optimization time of the $(1 + 1)$ EA for arbitrary linear functions.

We complement this upper bound by a lower bound of $(1 - o(1))en \ln n$. To do so, we showed that ONEMAX is the function easiest optimized by the $(1 + 1)$ EA. By this we extended a recent lower bound of $(1 - o(1))en \ln n$ for the expected optimization time on ONEMAX to all functions having a unique global optimum.

Our upper and lower bounds for the expected optimization times of the $(1 + 1)$ EA on arbitrary linear functions are relatively close. This raises the question if possibly all linear functions have the same expected optimization time of $(1 + o(1))en \ln n$.

Finally, we reviewed previous runtime analyses of the $(1 + 1)$ EA on the combinatorial problems of finding a minimum spanning tree, shortest path tree, or Euler tour in a graph. For all three cases, we exhibited the appearance of multiplicative drift and determined the leading constants in the bounds of the expected optimization times.

In the light of these natural occurrences of multiplicative drift, we are optimistic to see applications of multiplicative drift analysis in the near future.

Acknowledgements We like to thank Dirk Sudholt for pointing out that Theorem 11 holds for all pseudo-Boolean function with a unique global optimum rather than only for all linear functions.

References

1. Auger, A., Doerr, B. (eds.): Theory of Randomized Search Heuristics. Series on Theoretical Computer Science, vol. 1. World Scientific, Singapore (2011)
2. Baswana, S., Biswas, S., Doerr, B., Friedrich, T., Kurur, P., Neumann, F.: Computing single source shortest paths using single-objective fitness. In: FOGA '09: Proceedings of the 10th ACM Workshop on Foundations of Genetic Algorithms, pp. 59–66. ACM, New York (2009)

3. Doerr, B., Goldberg, L.A.: Drift analysis with tail bounds. In: PPSN '10: Proceedings of the 11th International Conference on Parallel Problem Solving from Nature, pp. 174–183. Springer, Berlin (2010)
4. Doerr, B., Goldberg, L.A.: Adaptive drift analysis. In: PPSN '10: Proceedings of the 11th International Conference on Parallel Problem Solving from Nature, pp. 32–41. Springer, Berlin (2010)
5. Doerr, B., Johannsen, D.: Adjacency list matchings—an ideal genotype for cycle covers. In: GECCO '07: Proceedings of the 9th Annual Genetic and Evolutionary Computation Conference, pp. 1203–1210. ACM, New York (2007)
6. Doerr, B., Johannsen, D.: Edge-based representation beats vertex-based representation in shortest path problems. In: GECCO '10: Proceedings of the 12th Annual Genetic and Evolutionary Computation Conference, pp. 759–766. ACM, New York (2010)
7. Doerr, B., Fouz, M., Witt, C.: Quasirandom evolutionary algorithms. In: GECCO '10: Proceedings of the 12th Annual Genetic and Evolutionary Computation Conference, pp. 1457–1464. ACM, New York (2010)
8. Doerr, B., Jansen, T., Sudholt, D., Winzen, C., Zarges, C.: Optimizing monotone functions can be difficult. In: PPSN '10: Proceedings of the 11th International Conference on Parallel Problem Solving from Nature, pp. 42–51. Springer, Berlin (2010)
9. Doerr, B., Johannsen, D., Winzen, C.: Drift analysis and linear functions revisited. In: CEC '10: Proceedings of the 2010 IEEE Congress on Evolutionary Computation, pp. 1967–1974. IEEE, New York (2010)
10. Doerr, B., Johannsen, D., Winzen, C.: Multiplicative drift analysis. In: GECCO '10: Proceedings of the 12th Annual Genetic and Evolutionary Computation Conference, pp. 1449–1456. ACM, New York (2010)
11. Droste, S., Jansen, T., Wegener, I.: On the analysis of the $(1 + 1)$ evolutionary algorithm. *Theor. Comput. Sci.* **276**(1–2), 51–81 (2002)
12. Giel, O., Lehre, P.K.: On the effect of populations in evolutionary multi-objective optimization. In: GECCO '06: Proceedings of the 8th Annual Genetic and Evolutionary Computation Conference, pp. 651–658. ACM, New York (2006)
13. Giel, O., Wegener, I.: Evolutionary algorithms and the maximum matching problem. In: STACS '03: Proceedings of the 20th Annual Symposium on Theoretical Aspects of Computer Science. Lecture Notes in Computer Science, vol. 2607, pp. 415–426. Springer, Berlin (2003)
14. Hajek, B.: Hitting-time and occupation-time bounds implied by drift analysis with applications. *Adv. Appl. Probab.* **14**(3), 387–403 (1982)
15. Happ, E., Johannsen, D., Klein, C., Neumann, F.: Rigorous analyses of fitness-proportional selection for optimizing linear functions. In: GECCO '08: Proceedings of the 10th Annual Genetic and Evolutionary Computation Conference, pp. 953–960. ACM, New York (2008)
16. He, J., Yao, X.: Drift analysis and average time complexity of evolutionary algorithms. *Acta Inform.* **127**(1), 51–81 (2001)
17. He, J., Yao, X.: Erratum to: Drift analysis and average time complexity of evolutionary algorithms [Artificial Intelligence 127 (2001) 57–85]. *Acta Inform.* **140**(1–2), 245–248 (2002)
18. He, J., Yao, X.: A study of drift analysis for estimating computation time of evolutionary algorithms. *Nat. Comput.* **3**(1), 21–35 (2004)
19. Jägersküpper, J.: A blend of Markov-chain and drift analysis. In: PPSN '08: Proceedings of the 10th International Conference on Parallel Problem Solving from Nature, pp. 41–51. Springer, Berlin (2008)
20. Jansen, T., Oliveto, P.S., Zarges, C.: On the analysis of the immune-inspired B-Cell Algorithm for the vertex cover problem. In: ICARIS '11: Proceedings of the 10th International Conference on Artificial Immune Systems. Lecture Notes in Computer Science, vol. 6825, pp. 117–131. Springer, Berlin (2011)
21. Kano, M.: Maximum and k th maximal spanning trees of a weighted graph. *Combinatorica* **7**(2), 205–214 (1987)
22. Mühlenbein, H.: How genetic algorithms really work: mutation and hill-climbing. In: PPSN '92: Proceedings of the 2nd International Conference on Parallel Problem Solving from Nature, pp. 15–25 (1992)
23. Neumann, F., Wegener, I.: Randomized local search, evolutionary algorithms, and the minimum spanning tree problem. *Theor. Comput. Sci.* **378**(1), 32–40 (2007)
24. Neumann, F., Oliveto, P.S., Witt, C.: Theoretical analysis of fitness-proportional selection: landscapes and efficiency. In: GECCO '09: Proceedings of the 11th Annual Genetic and Evolutionary Computation Conference, pp. 835–842. ACM, New York (2009)

25. Oliveto, P.S., Witt, C.: Simplified drift analysis for proving lower bounds in evolutionary computation. *Algorithmica* **59**(3), 369–386 (2011)
26. Scheder, D., Welzl, E.: Private communications (2008)
27. Witt, C.: Tight bounds on the optimization time of the $(1 + 1)$ EA on linear functions. In: STACS '12: Proceedings of the 29th Annual Symposium on Theoretical Aspects of Computer Science (2012, to appear)